2026 생성형AI활용 프로젝트 기획 발표

근거로 확인하는 AI 변경 요약

Git 없이 쓰는 프로젝트 버전 비교 도구

최종.zip → 진짜최종.zip → **무엇이 바뀌었나?**

팀 근성

지용준(조장) · 박민제 · 나현준 · 최림정 · 신동윤

배경 및 필요성

버전은 쌓이는데, 무엇이 바뀌었는지는 알기 어렵다

과제.zip
10/06

→

최종.zip
10/13

→

최종수정.zip
10/17

→

진짜최종.zip
10/20

현실

입문자와 소규모 팀은 파일을 복사하거나
zip · 클라우드로 버전을 관리하는 경우가 많음

문제

어떤 파일이 바뀌었는지, 무엇이 달라졌는지
매번 직접 열어서 비교해야 함

영향

코드를 모르는 기획 · 디자인 팀원은
진행 상황을 파악하기 더 어려움

배경 및 필요성

기존 방법의 한계

Git

변경 추적은 정확하지만
명령어와 개념을 익혀야 해서
입문자에게 진입 장벽이 높음

diff 비교 도구

줄 단위 차이만 보여줘서
파일이 많으면 전체적으로
무엇이 바뀌었는지 파악하기 어려움

AI 요약

읽기 쉽지만 근거가 없고
실제로 없는 변경을 지어내는
환각이 있어 그대로 믿기 어려움

필요한 것 : AI 요약의 편리함 + 직접 확인할 수 있는 근거

핵심 아이디어

사실은 엔진이, 해석은 AI가

사실 · Python 엔진

파일 목록, 변경 유형, 줄 개수, 이동 여부

LLM을 사용하지 않고 엔진이 계산해 리포트에 그대로 넣음
= 사실 정보 오류 0%를 구조적으로 보장

해석 · LLM

이 변경이 무슨 의미인지 설명하는 문장

문장마다 근거 diff 조각을 반드시 지정하고
시스템이 근거와 자동 대조해 불일치를 경고

목표 및 기대효과

네 가지 목표

01 **변경 자동 분류**

폴더 · zip 두 개를 비교해
추가 · 삭제 · 수정 · 이동을 자동 분류

02 **사실 정보 오류 0%**

파일 목록 · 변경 · 위치는
엔진이 생성해 구조적으로 보장

03 **근거 연결과 자동 경고**

AI 문장마다 근거 diff를 연결하고
불일치하면 경고 표시

04 **사용자별 리포트**

개발자용 상세 리포트와
비개발자용 쉬운 리포트를 따로 제공

기대효과 변경 확인 시간 단축 · 비개발 팀원도 진행 상황 파악 · AI 요약을 근거로 검토 가능

주요 내용 요약

개발 범위

입력

- 폴더 ↔ 폴더
- zip ↔ zip
- 폴더 ↔ zip 혼합
- 구조가 바뀌어도 내용으로 짝 찾기

지원 파일

- 코드 · txt · md
- hwp · hwpx (본문 텍스트)
- docx (본문 텍스트)
- 바이너리는 "변경됨"만 표시

범위 밖

- 스캔 pdf (OCR)
- 이미지 내용 비교
- 글꼴 · 서식 변경 감지
- 모델 직접 학습

사용 형태 명령어 한 줄로 실행 → HTML 리포트 생성 → 이후 웹 화면(끌어다 놓기)으로 확장

적용할 AI 기술

생성형 AI(LLM) + 자연어 처리

모델을 직접 학습시키지 않음 · 이미 학습된 LLM을 설계로 신뢰성 있게 활용

구조화 출력

자유 문장이 아닌
정해진 JSON 형식으로만 답하게 하고
형식이 틀리면 자동 재요청

계층적 요약

파일 → 폴더 → 전체 순서로 요약해
입력 길이 한계를 넘는 대규모 변경도 처리

근거 지정 프롬프트

문장에 근거 diff 조각 번호를 달게 하고
숫자는 쓰지 않도록 지시

모델 구조 및 데이터 흐름

6단계 처리 흐름

1
입력
폴더 · zip 두 개 (이전 / 이후)

→

2
노이즈 제거
캐시 · 빌드 · 비밀 파일 제외

→

3
비교 엔진
추가 · 삭제 · 수정· 이동 분류, diff 통계

→

4
AI 요약
파일→폴더→전체, 문장마다 근거

→

5
근거 검증
근거 · 식별자 · 숫자 자동 대조

→

6
리포트
개발자용 / 비개발자 전용 HTML

규칙 기반 처리 (사실 · 항상 같은 결과) LLM 처리 (해석)

LLM이 관여하는 단계는 4단계 하나뿐, 그 결과도 5단계에서 다시 검증

주요 알고리즘

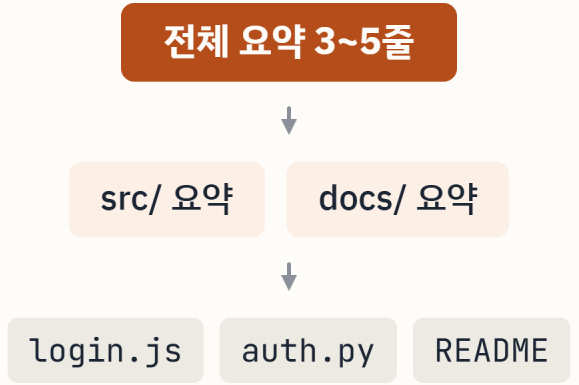
파일 매칭과 계층 요약

파일 매칭 (폴더 구조가 바뀌어도)

1	같은 경로끼리 짝짓기
2	내용 해시가 같으면 → 이동
3	이름 · 내용 유사하면 → 이동 + 수정
4	폴더 단위로 묶기 → "frontend/ → client/"

공통 파일이 거의 없으면 **같은 프로젝트가 맞는지 확인** 경고 후 중단

계층 요약 (대규모 변경 대응)



파일 요약의 핵심 키워드 · 함수명을 상위 단계로 전달해 정보 손실 방지

근거 검증

AI 문장을 근거와 자동 대조

LLM 출력 예시 (JSON)

```
{
  "file": "src/login.js",
  "summary": "로그인 실패 시
    validatePassword()로
    비밀번호 형식 검사 추가",
  "evidence": ["H12"]
}
```

H12 = src/login.js 40~58줄 변경 조각 (엔진이 번호 부여)

자동 검증 규칙

- ✓ 근거 H12가 실제로 존재하는가
- ✓ 문장의 식별자(validatePassword)가 H12 안에 있는가
- ✓ 언급한 파일이 실제 변경 목록에 있는가
- ✓ 숫자를 썼는가 (숫자는 엔진만 쓸 수 있음)

⚠ 하나라도 어긋나면 ⚠ 경고 뱃지 + 근거 위치로 바로 이동

데이터 수집 및 전처리

테스트 데이터와 전처리

데이터 출처

1. 팀원들의 예전 과제 버전 파일

2. GitHub 공개 저장소의 커밋 두 개를
   스냅샷 쌍으로 변환

3. 직접 만든 테스트 세트
   (hwp · docx 수정본, 폴더 이동)

② 는 Git 기록이 곧 정답
→ 평가용 정답 데이터 자동 확보

전처리

- 노이즈 제외
  (빌드 산출물, 캐시, node_modules)
- 인코딩 판별 (cp949 / utf-8)
- 문서 파일 본문 텍스트 추출
- zip의 최상위 폴더 정리

윤리적 고려

- 개인정보 · 남의 비공개 코드는
  LLM에 보내지 않음
- API 키 · 비밀번호 파일(.env 등)은
  전송 전 자동 제외
- 공개 저장소 사용 시 라이선스 확인

구현 도구 및 환경

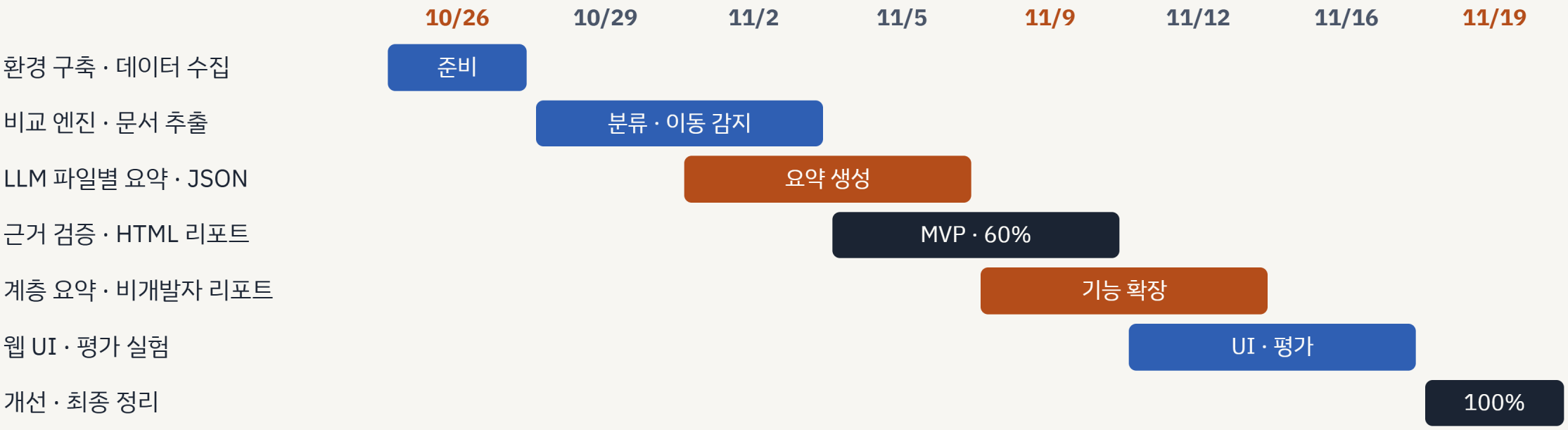
무료 도구로 구성한 개발 환경

구분	도구
언어	Python 3.12
비교·분석	difflib, hashlib, rapidfuzz(유사도), pathspec(제외 규칙)
문서 추출	olefile(hwp), python-docx, charset-normalizer(인코딩)
LLM	Gemini Flash 무료 티어 · 예비: Groq, Ollama(로컬)
출력 검증·리포트	pydantic(JSON 형식 검증), jinja2(HTML), Streamlit(웹 UI)
개발 도구	VS Code, Claude Code, GitHub
하드웨어	개인 노트북 (모델 학습이 없어 GPU 불필요)

LLM 호출부를 분리해 설정만 바꾸면 모델 교체 가능 · 개발 중에는 가짜 응답(mock)으로 비용 없이 테스트

3.1 프로젝트 일정표

10/26 ~ 11/20 일정과 마일스톤



10/26 기획 발표

11/9 중간보고 · 핵심 기능 60% (MVP 시연)

11/20 기능 100% 완성

3.2 팀 역할 분담

조장 중심 개발, 조원은 검수와 지원

지용준

조장 · 개발 총괄

- 기획 · 전체 구조 설계
- 비교 엔진 · 문서 추출
- LLM 요약 · 근거 검증 로직
- HTML 리포트 · 웹 UI
- 평가 실험 · 문서 작성

나현준

발표

기획·중간·최종 발표 진행

박민제

데이터 제공

테스트용 프로젝트·문서 파일 제공

최림정

검수

리포트 결과 확인, 오류 제보

신동윤

보조

자료 조사, 발표 자료 정리

위험 요소 및 대응 방안

예상 위험과 대응

구분	위험	대응
기술	LLM 환각	사실 · 해석 분리, 근거 검증, 경고 표시
기술	대규모 변경 시 입력 길이 초과	계층 요약, 중요도 순 처리
기술	hwp 파싱 실패 (암호화 · 배포용)	추출 실패 시 변경됨만 표시
비용	무료 API 요청 제한	대기 · 재시도 로직, 모델 교체 구조, mock 개발
일정	기능 과다로 일정 지연	11/9 중간보고까지 MVP 우선, 나머지는 선택 기능
협업	개발이 한 명에게 집중되는 병목	조원이 데이터 제공 · 검수 · 발표를 분담, 주 1회 진행 공유

성능 평가 지표

측정 가능한 지표만 설정

평가 대상	지표	목표 (예시)
변경 분류	Precision · Recall · F1 (추가 · 삭제 · 수정 · 이동)	F1 0.95 이상
이동 감지	Recall	0.9 이상
사실 정보	오류율	0% (구조적 보장)
근거 검증	주입 오류 탐지율 (Recall)	80% 이상
근거 검증	오탐률 (정상 문장에 경고)	10% 이하
요약 품질	사람 판정 정확도 (30~50문장 샘플)	측정 결과 보고

오류 주입 평가

정상 요약에 틀린 파일명 · 가짜 함수명 · 엉뚱한 근거를
자동으로 넣고 검증기가 몇 개를 잡는지 측정

BLEU를 쓰지 않는 이유

정답 요약문이 없는 과제라 의미가 없음.
목표 수치는 1차 실험 후 확정

사용자 테스트와 개선 계획

사용자 테스트 설계

대상 · 수강생 5명 내외 (개발자 · 비개발자 혼합)

시기 · 11/12 ~ 11/18 (기능 확장 후)

과제 · 같은 스냅샷 쌍에서 "주요 변경 3가지 찾기"

비교 · A 직접 비교 vs B 리포트 사용

측정 · 소요 시간, 정답률, SUS 사용성 설문(10문항), 자유 의견 등등

결과에 따른 개선

놓친 오류 유형 → 검증 규칙 추가, 프롬프트 수정

오탐이 많음 → 식별자 대조 기준 완화

비개발자가 이해 못 한 문장 → 리포트 문구 템플릿 개선

시간 단축 효과 미미 → 요약 길이 · 리포트 구성 재설계

기대효과, 한계, 그리고 다음 단계

5.1 기대효과

- **교육** · 입문자가 Git 없이 버전 관리 개념을 익힘
- **협업** · 비개발 팀원과 진행 상황 공유
- **기술** · 근거 기반 AI 출력의 신뢰성 설계 사례

5.2 한계점

- 의미 오류는 완전히 잡을 수 없음 (최종 판단은 사용자)
- 서식 · 이미지 · 스캔 문서는 비교 불가
- 무료 API 제한으로 대규모 처리 속도 저하

5.3 후속 방향

- 2차 LLM 교차 검증으로 의미 오류 보강
- Git · Google Drive 버전 기록 연동
- VS Code 확장 또는 웹 서비스화
- 서식 · 이미지 변경 비교 지원

참고 자료

논문

Ji, Z. et al. (2023). Survey of Hallucination in Natural Language Generation. *ACM Computing Surveys*, 55(12).

Jia, Y. & Harman, M. (2011). An Analysis and Survey of the Development of Mutation Testing. *IEEE Transactions on Software Engineering*, 37(5).

비교 도구

Git 공식 문서 (git diff, rename 감지 -M 옵션) · MemoV

공식 문서

- Python 표준 라이브러리 : difflib, hashlib, zipfile
- Google Gemini API : 구조화 출력(Structured Output)
- pydantic · Jinja2 · Streamlit 공식 문서
- olefile · python-docx 공식 문서
- 한글 문서 파일 형식(HWP 5.0) 공개 문서 · 한글과컴퓨터

근거로 확인하는 AI 변경 요약

감사합니다

사실은 엔진이, 해석은 AI가, 판단은 근거로